ESP32 UWB Pro with Display

Annotated Source Code · DW1000 Anchor Firmware

This document presents the full source code for the ESP32 UWB Anchor sketch, with inline comments explaining every section. Anchors are fixed reference points deployed around a space; they respond to ranging requests from mobile tags and enable position calculation. Flash multiple boards with this sketch, uncommenting a different `ANCHOR_ADD` line on each one. Unlike the Tag sketch, anchors require no linked-list – they simply listen, respond, and log.

■ File Header & Overview

```
/*
 * ESP32 UWB Pro with Display – ANCHOR firmware
 *
 * This sketch runs on an ESP32 paired with a DW1000 UWB radio module and a
 * 128×64 SSD1306 OLED.  The board acts as a UWB *anchor*: a fixed reference
 * point that responds to ranging requests from mobile *tags*.
 *
 * In a typical deployment you flash multiple boards with this sketch, each
 * with a different ANCHOR_ADD address (::00, ::01, ::02, ::03 …).  Tags
 * range against all visible anchors simultaneously to compute position.
 *
 * Key difference from the Tag sketch: no linked-list is needed here because
 * the anchor does not track distance itself – it just responds and reports.
 */
```

■ Includes & Libraries

```
#include <SPI.h>            // Hardware SPI bus – communicates with the DW1000 chip
#include "DW1000Ranging.h"  // Makerfabs/thotro UWB ranging library
#include <Wire.h>           // I²C bus – used by the OLED display
#include <Adafruit_GFX.h>           // Core Adafruit graphics primitives
#include <Adafruit_SSD1306.h>       // Driver for the 128×64 SSD1306 OLED
```

■ Anchor Address Selection

```
// Each anchor in the network must have a unique EUI-64 address.
// Uncomment exactly ONE of these lines to assign this board its anchor ID.
// The last byte (00–03) is the conventional way to number anchors 0–3.
//
// Typical placement for a 2-D positioning system:
```

```
//   ::00  top-left corner
//   ::01  top-right corner
//   ::02  bottom-right corner
//   ::03  bottom-left corner  ← currently active
//
//#define ANCHOR_ADD "26:00:5B:D5:A9:9A:E2:9C"   // Anchor 0
//#define ANCHOR_ADD "26:01:5B:D5:A9:9A:E2:9C"   // Anchor 1
//#define ANCHOR_ADD "26:02:5B:D5:A9:9A:E2:9C"   // Anchor 2
#define ANCHOR_ADD "26:03:5B:D5:A9:9A:E2:9C"     // Anchor 3 – ACTIVE
```

■ Pin Configuration

```
// SPI bus pins – connect to the DW1000 module
#define SPI_SCK  18   // SPI clock
#define SPI_MISO 19   // Master-in / slave-out
#define SPI_MOSI 23   // Master-out / slave-in
// DW1000 control pins
#define UWB_RST 27    // Active-low hardware reset line
#define UWB_IRQ 34    // Interrupt line – signals the ESP32 when data is ready
#define UWB_SS  21    // SPI chip-select (active low)
// I²C pins for the OLED
#define I2C_SDA 4
#define I2C_SCL 5
// OLED instance: 128 px wide, 64 px tall, on Wire, no external RESET pin (-1).
Adafruit_SSD1306 display(128, 64, &Wire;, -1);
```

■ setup() – One-time Initialisation

```
void setup() {
    Serial.begin(115200);  // Open serial console for debug output
    // Initialise I²C with the custom SDA/SCL pins defined above.
    Wire.begin(I2C_SDA, I2C_SCL);
    delay(1000);  // Give the OLED time to power up after supply is stable
    // Start the SSD1306 driver at I²C address 0x3C (default for most modules).
    // If the display is not found, halt with an infinite loop rather than
    // continuing with a blank/broken UI.
    if (!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
        Serial.println(F("SSD1306 allocation failed"));
        for (;;);
    }
    display.clearDisplay();
    logoshow();   // Show splash screen (includes this anchor's address string)
    // Bring up the SPI bus, then hand the three DW1000 control pins to the
    // ranging library.  Argument order: Reset, ChipSelect, IRQ.
    SPI.begin(SPI_SCK, SPI_MISO, SPI_MOSI);
```

```
    DW1000Ranging.initCommunication(UWB_RST, UWB_SS, UWB_IRQ);
    // Register event callbacks – the library fires these automatically:
    DW1000Ranging.attachNewRange(newRange);          // A tag completed a range measurement
    DW1000Ranging.attachBlinkDevice(newBlink);       // A tag sent its first discovery blink
    DW1000Ranging.attachInactiveDevice(inactiveDevice); // A tag has gone silent / timed out
    // Start in ANCHOR mode.  The 'false' argument disables the range filter
    // (smoothing); set to 'true' if you want the library to low-pass the
    // distance values.  MODE_LONGDATA_RANGE_LOWPOWER maximises range at the
    // cost of data rate – ideal for static anchor deployments.
    DW1000Ranging.startAsAnchor(ANCHOR_ADD, DW1000.MODE_LONGDATA_RANGE_LOWPOWER, false);
}
```

■ loop() – Main Event Loop

```
void loop() {
    // Hand control to the DW1000 ranging library every iteration.
    // It drives the UWB state machine and fires callbacks when events occur.
    // The anchor has no display refresh logic here – it simply listens and
    // responds to tag ranging requests continuously.
    DW1000Ranging.loop();
}
```

■ DW1000 Callbacks

```
// Called each time a tag completes a two-way ranging exchange with this anchor.
// The anchor itself does not move, so this is purely for serial logging –
// the *tag* is responsible for recording the distance and computing position.
void newRange() {
    Serial.print("from: ");
    Serial.print(DW1000Ranging.getDistantDevice()->getShortAddress(), HEX);
    Serial.print("\t Range: ");
    Serial.print(DW1000Ranging.getDistantDevice()->getRange());
    Serial.print(" m");
    Serial.print("\t RX power: ");
    Serial.print(DW1000Ranging.getDistantDevice()->getRXPower());
    Serial.println(" dBm");
}
// Called when a tag sends its initial UWB 'blink' frame to announce itself.
// This is the discovery phase before full two-way ranging begins.
// The short address is a 16-bit handle derived from the tag's full EUI-64.
void newBlink(DW1000Device *device) {
    Serial.print("blink; 1 device added ! -> ");
    Serial.print(" short:");
    Serial.println(device->getShortAddress(), HEX);
}
// Called when a previously active tag has not communicated for the library's
```

```
// inactivity timeout period.  Logged here for diagnostic purposes.
void inactiveDevice(DW1000Device *device) {
    Serial.print("delete inactive device: ");
    Serial.println(device->getShortAddress(), HEX);
}
```

■ OLED Splash Screen

```
// Display a startup splash screen showing the anchor label and its full
// address string.  Showing the address is especially useful when you have
// several identical-looking anchor boards deployed in a space – you can
// immediately identify which board you're looking at.
void logoshow(void) {
    display.clearDisplay();

    // Line 1 & 2 – large text: "Makerfabs" / "UWB Anchor"
    display.setTextSize(2);              // 2× scale = 16 px tall characters
    display.setTextColor(SSD1306_WHITE);
    display.setCursor(0, 0);
    display.println(F("Makerfabs"));
    display.println(F("UWB Anchor"));

    // Line 3 – small text: full EUI-64 address string (e.g. 26:03:5B:…)
    // Printed at y=40 to sit just below the two large lines (2 × ~20 px).
    display.setTextSize(1);              // 1× scale = 8 px tall characters
    display.setCursor(0, 40);
    display.println(ANCHOR_ADD);         // Uses the #define set at the top of the file

    // Commit the frame buffer to the physical display.
    // Unlike the tag sketch, there is no loop refresh – the splash stays
    // on screen permanently since the anchor has nothing else to show.
    display.display();
}
```

End of annotated source. Hardware: Makerfabs ESP32 UWB Pro · Library: DW1000Ranging / thotro